NSDC Data Science Projects

Project: Ad Targeting

Name: Mukaila Rafiu

Description:

This project aims to analyze advertising strategies to understand, and potentially improve, engagement. We'll walk through each step of the data science process, from problem definition to insights and data-driven recommendations.

Introduction

In this contemporary digital age, online advertisement plays a key role in Business revenue generation via marketing, however, umderstanding usher engagement with the adverts remains a big question for businesses. This project is particularly relevant in the context of digital marketing, where companies aim to optimize their ad strategies to increase engagement and conversion rates. By applying data science techniques, we can uncover patterns in user behavior that help marketers make data-driven decisions. in this project, we will utilize advertising data from a marketing company to predict whether a user will click on an advertisement or not

Milestone 1: Data Loading and Preprocessing

In [2]:
```
#Load packages
import dtale
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
```

In [3]:
```
#Load Data
df = pd.read_csv("C://Users//Mukaila Rafiu//Downloads//advertising_ef.csv")
df
```

Out[3]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	City	Gender	Country
0	68.95	35.0	61833.90	256.09	Cloned 5thgeneration orchestration	Wrightburgh	Female	Tunisia
1	NaN	31.0	68441.85	193.77	Monitored national standardization	West Jodi	Male	Nauru
2	69.47	26.0	59785.94	236.50	Organic bottom-line service-desk	Davidton	Female	San Marino
3	74.15	29.0	54806.18	245.89	Triple-buffered reciprocal time-frame	West Terrifurt	Male	Italy
4	68.37	35.0	73889.99	225.58	Robust logistical utilization	South Manuel	Female	Iceland
...	...	...	...	...	...	...	...	...
1004	72.97	30.0	71384.57	208.58	Fundamental modular algorithm	Duffystad	Male	Lebanon
1005	51.30	45.0	67782.17	134.42	Grass-roots cohesive monitoring	New Darlene	Male	Bosnia and Herzegovina
1006	51.63	51.0	42415.72	120.37	Expanded intangible solution	South Jessica	Male	Mongolia
1007	55.55	19.0	41920.79	187.95	Proactive bandwidth-monitored policy	West Steven	Female	Guatemala
1008	45.01	26.0	29875.80	178.35	Virtual 5thgeneration emulation	Ronniemouth	Female	Brazil

1009 rows × 10 columns

In [4]:

```
#Checking data type
df.dtypes
```

Out[4]:
```
Daily Time Spent on Site    float64
Age                         float64
Area Income                 float64
Daily Internet Usage        float64
Ad Topic Line                object
City                         object
Gender                       object
Country                      object
Timestamp                    object
Clicked on Ad                 int64
dtype: object
```

In [5]:

In [6]:
```
#Checking data columns
df.columns
```

Out[6]:
```
Index(['Daily Time Spent on Site', 'Age', 'Area Income',
       'Daily Internet Usage', 'Ad Topic Line', 'City', 'Gender', 'Country',
       'Timestamp', 'Clicked on Ad'],
      dtype='object')
```

Here, the main variable to focus on is “Clicked on Ad.” This variable has two possible values: 0 or 1. A value of 0 means that the user did not click on the advertisement, while a value of 1 indicates that the user did click on the advertisement.

In [7]:
```
# Calculate the number of missing values for each column
missing_values = df.isnull().sum()

print("Number of missing values per column:", missing_values)
```

```
Number of missing values per column: Daily Time Spent on Site     4
Age                         11
Area Income                 11
Daily Internet Usage         4
Ad Topic Line                0
City                        11
Gender                       0
Country                     13
Timestamp                    0
Clicked on Ad                0
dtype: int64
```

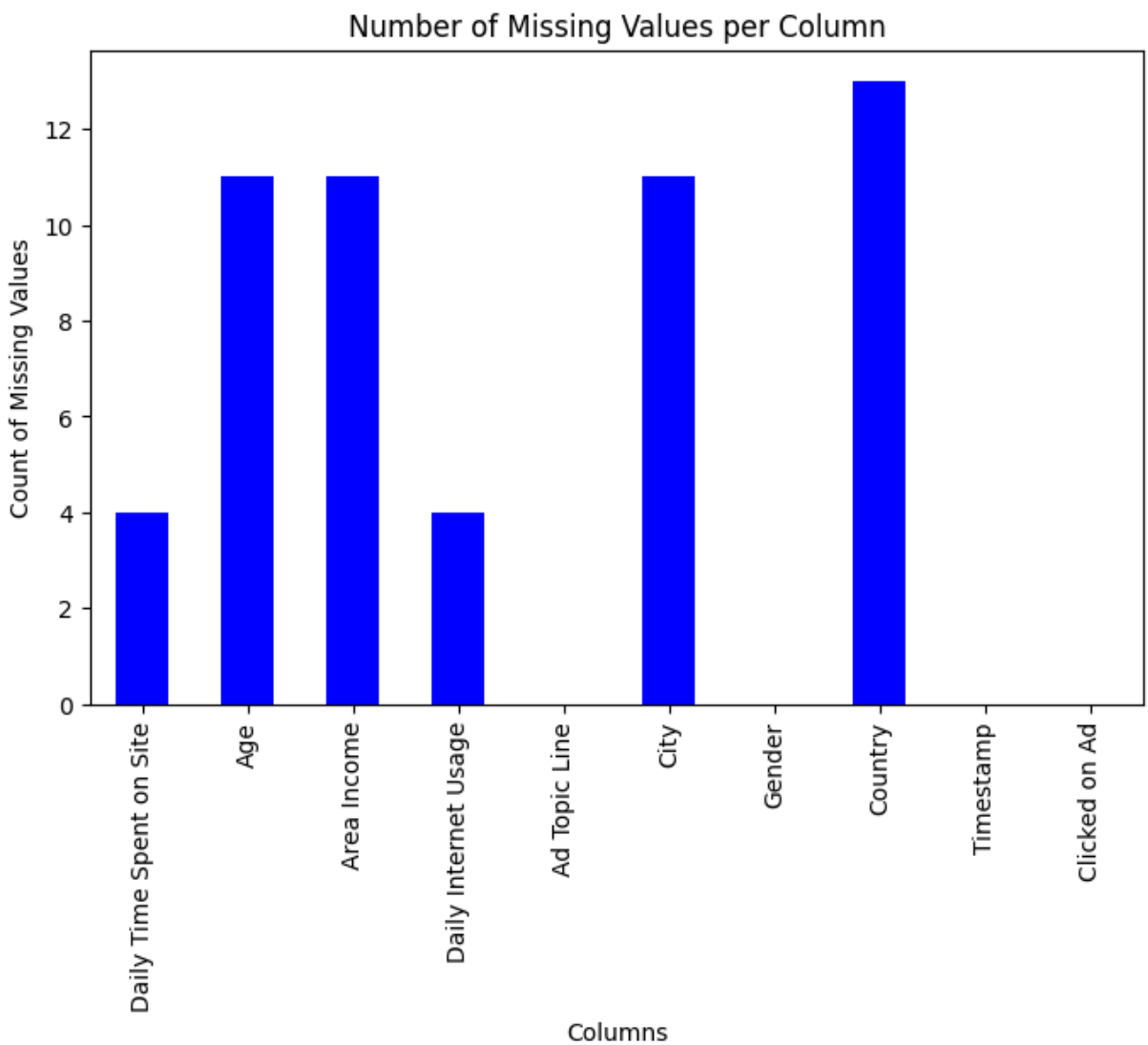
Country, City, Age, and Area Income dominate the number of missing values

In [8]:
```
# Percentage of missing values in each column
null_percent = df.isnull().mean()*100
print(null_percent)
```

```
Daily Time Spent on Site    0.396432
Age                         1.090188
Area Income                 1.090188
Daily Internet Usage        0.396432
Ad Topic Line               0.000000
City                        1.090188
Gender                      0.000000
Country                     1.288404
Timestamp                   0.000000
Clicked on Ad               0.000000
dtype: float64
```

In [9]:

```
#Plot the missing values as a bar chart
plt.figure(figsize=(8, 5))
missing_values.plot(kind='bar', color='blue')
plt.title('Number of Missing Values per Column')
plt.xlabel('Columns')
plt.ylabel('Count of Missing Values')

plt.xticks(rotation=90)
plt.show()
```



Best practice in data science:

5–10% missing is acceptable depending on context

10% or more missing is often remove or carefully impute

Drop columns with more than 10% missing valuesabs This keeps only columns with less than 10% missing values. df = df.loc[:, df.isnull().mean() < 0.10]

In [12]:
```
# Fill missing values with mean for the integer columns for simplicity
# Numeric columns
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Area Income'].fillna(df['Area Income'].mean(), inplace=True)

# Categorical columns
df['City'].fillna(df['City'].mode()[0], inplace=True)
df['Country'].fillna(df['Country'].mode()[0], inplace=True)
```

In [13]:
```
#Checking the missing values after filling them up
df.isnull().sum()
```

Out[13]:
```
Daily Time Spent on Site    4
Age                         0
Area Income                 0
Daily Internet Usage        4
Ad Topic Line               0
City                        0
Gender                      0
Country                     0
Timestamp                   0
Clicked on Ad               0
dtype: int64
```

Strategy for Handling the Missing Values

Dropping the rows with missing values is not ideal in this case because we do not have enough data. Dropping those rows would lead to a significant loss in information. Instead, we use the following ways to keep that information while dealing with missing values:

- **Numerical Columns:** Imputing with the mean - it ensures that we retain all rows while maintaining the overall distribution of the data.
- **Categorical Columns:** Filling with 'Unknown' allows us to keep information about rows where location data (`'City'` or `'Country'`) was missing, which could be useful for analysis or modeling.df

Cleaning the dataset

Looking at the data, we can see that we need to convert the `Timestamp` column into a more usable format and extract additional features such as `Hour`, `Day`, and `Month`.

- The first step is to convert the `Timestamp` column from a `string` format to a proper `datetime` format using `pd.to_datetime()`.
- After converting the `Timestamp` column to `datetime`, we can extract the desired features.

In [16]:
```
# Convert 'Timestamp' to datetime
df['Timestamp'] = pd.to_datetime(df['Timestamp'], format='mixed')

# Extract additional features from 'Timestamp'
df['Hour'] = df['Timestamp'].dt.hour
df['Day'] = df['Timestamp'].dt.day
df['Month'] = df['Timestamp'].dt.month
```

In [17]:
```
# Print head to see the new features
df.head()
```

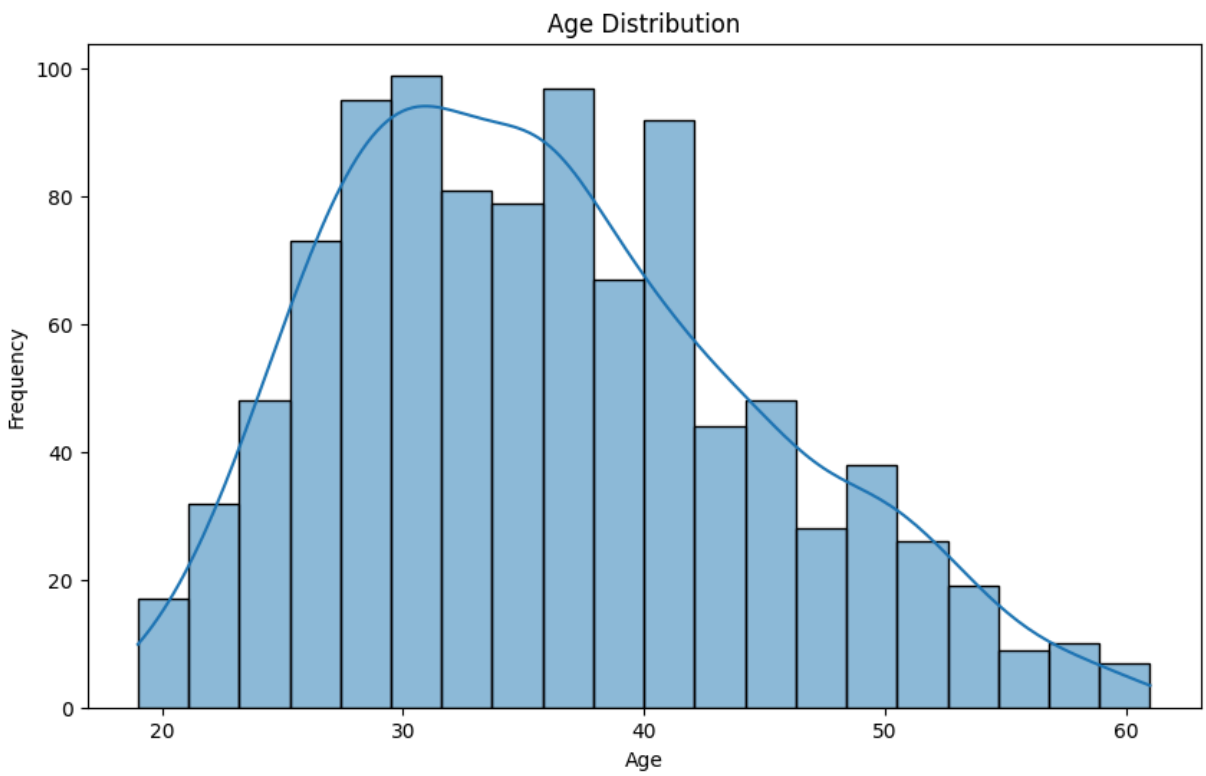
Out[17]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	City	Gender	Country	Timestar
0	68.95	35.0	61833.90	256.09	Cloned 5thgeneration orchestration	Wrightburgh	Female	Tunisia	2016-03- 00:53
1	NaN	31.0	68441.85	193.77	Monitored national standardization	West Jodi	Male	Nauru	2016-04- 01:39
2	69.47	26.0	59785.94	236.50	Organic bottom-line service-desk	Davidton	Female	San Marino	2016-03- 20:35
3	74.15	29.0	54806.18	245.89	Triple-buffered reciprocal time-frame	West Terrifurt	Male	Italy	2016-10- 02:31
4	68.37	35.0	73889.99	225.58	Robust logistical utilization	South Manuel	Female	Iceland	2016-03- 03:36

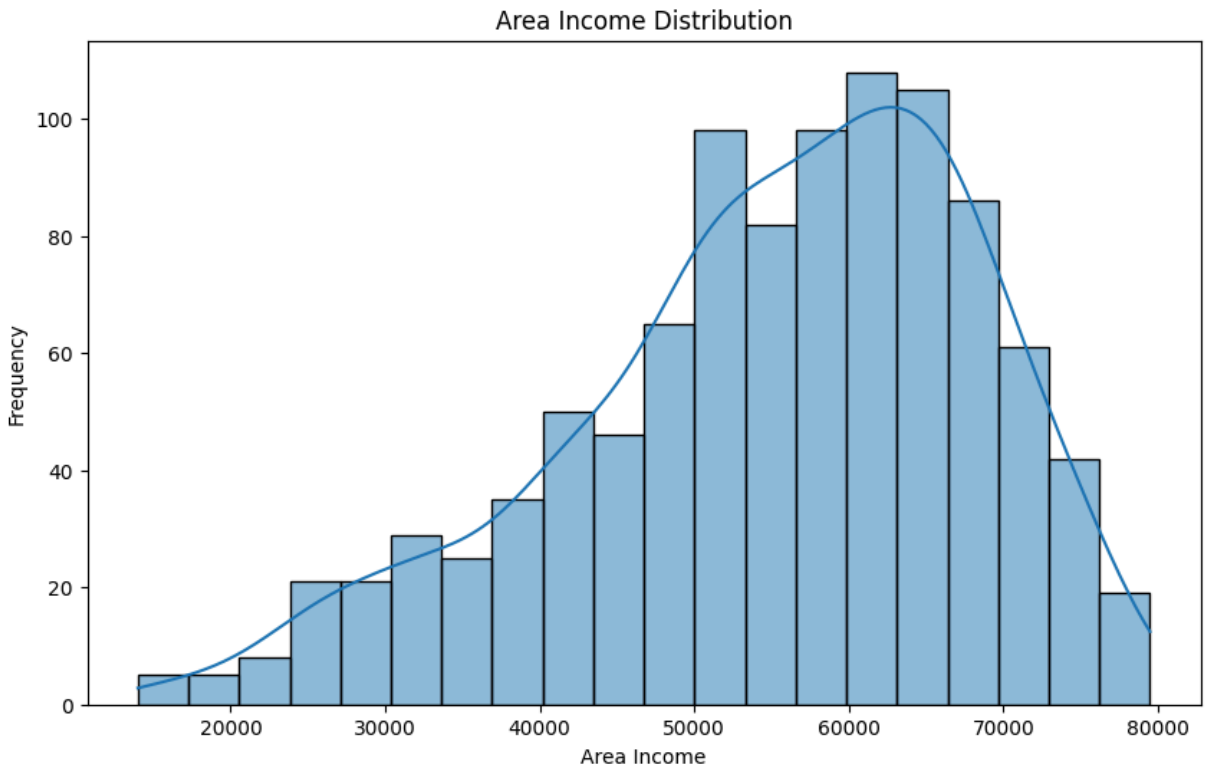
The data looks clean and ready for analysis

Milestone 2: Exploratory Data Visualization

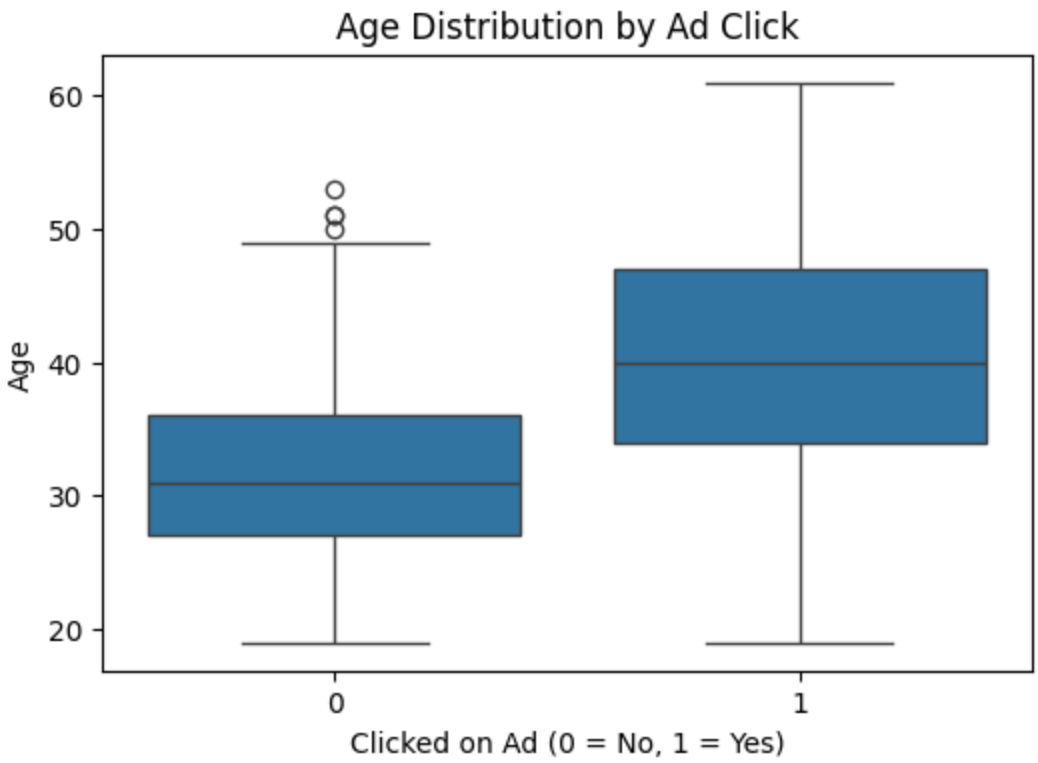
In [18]:
```
# Age Distribution Histogram
plt.figure(figsize=(10,6))
sns.histplot(df['Age'], bins=20, kde=True)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



In [19]:
```
# Area Income Distribution Histogram
plt.figure(figsize=(10,6))
sns.histplot(df['Area Income'], bins=20, kde=True)
plt.title('Area Income Distribution')
plt.xlabel('Area Income')
plt.ylabel('Frequency')
plt.show()
```

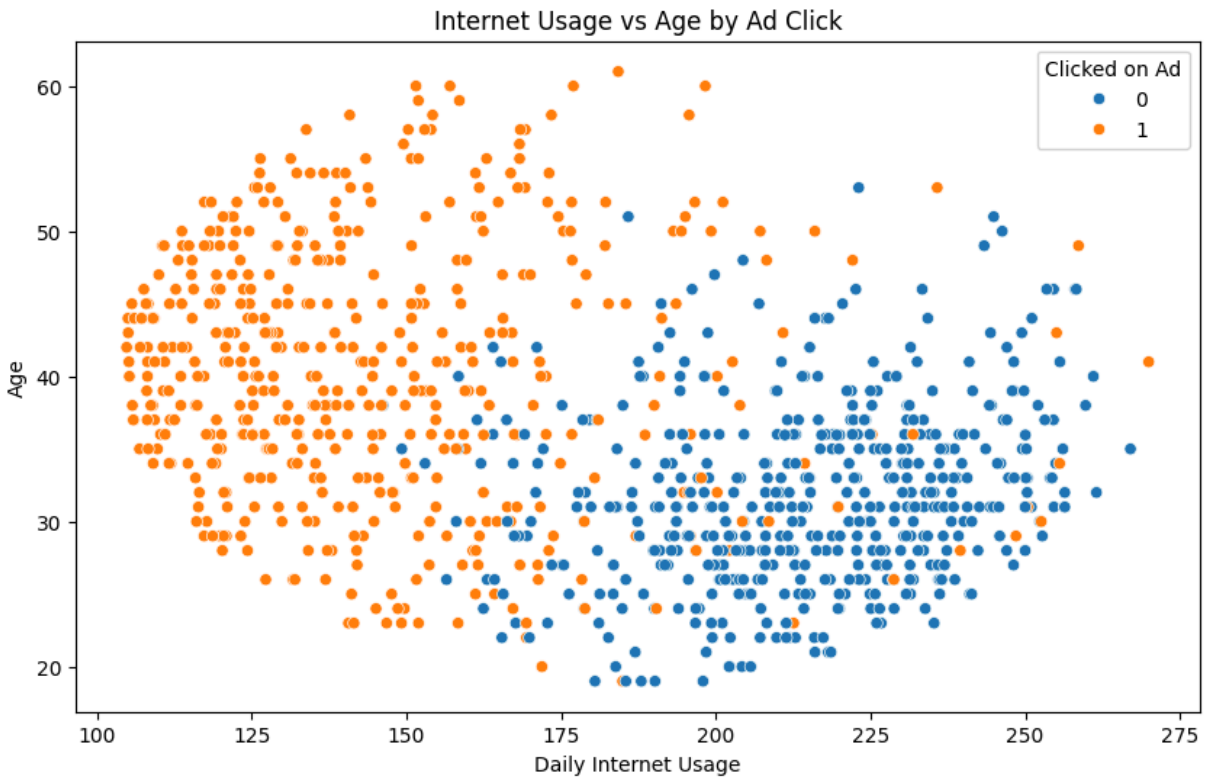


In [20]:
```
# Age vs Ad Click
# To check people who click ads tend to be older or younger
plt.figure(figsize=(6,4))

sns.boxplot(x='Clicked on Ad', y='Age', data=df)

plt.title('Age Distribution by Ad Click')
plt.xlabel('Clicked on Ad (0 = No, 1 = Yes)')
plt.ylabel('Age')

plt.show()
```



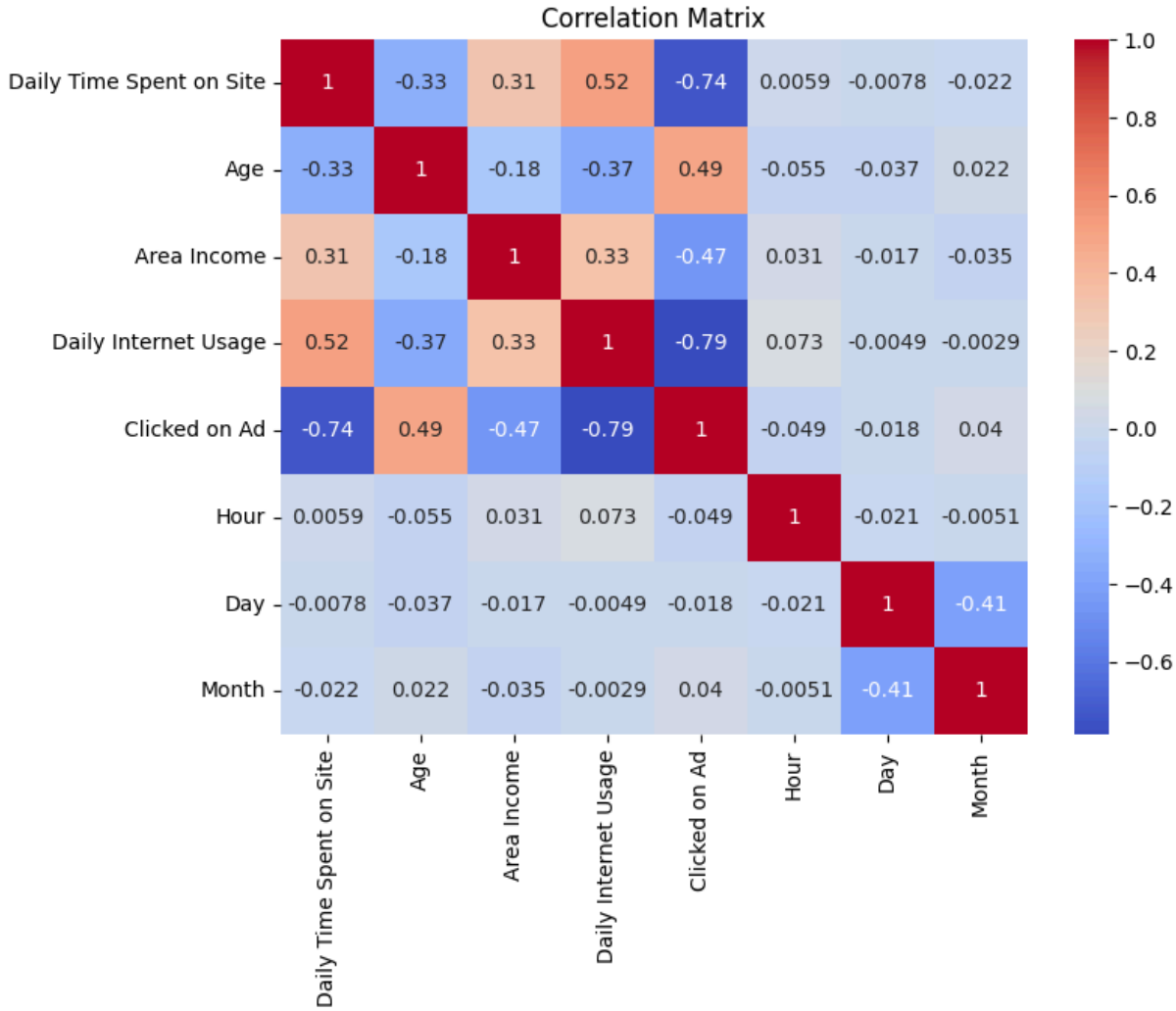
Ushers between the age of 35 to 45 engage in ad clicking whiles usher between the age of 28 to 37 do not click ads

In [21]:
```
# Daily Internet Usage vs Ad Click
# strongest predictors in this dataset
plt.figure(figsize=(10,6))
sns.scatterplot(x='Daily Internet Usage', y='Age', hue='Clicked on Ad', data=df)
plt.title('Internet Usage vs Age by Ad Click')
plt.xlabel('Daily Internet Usage')
plt.ylabel('Age')
plt.show()
```



This confirms the boxplot result

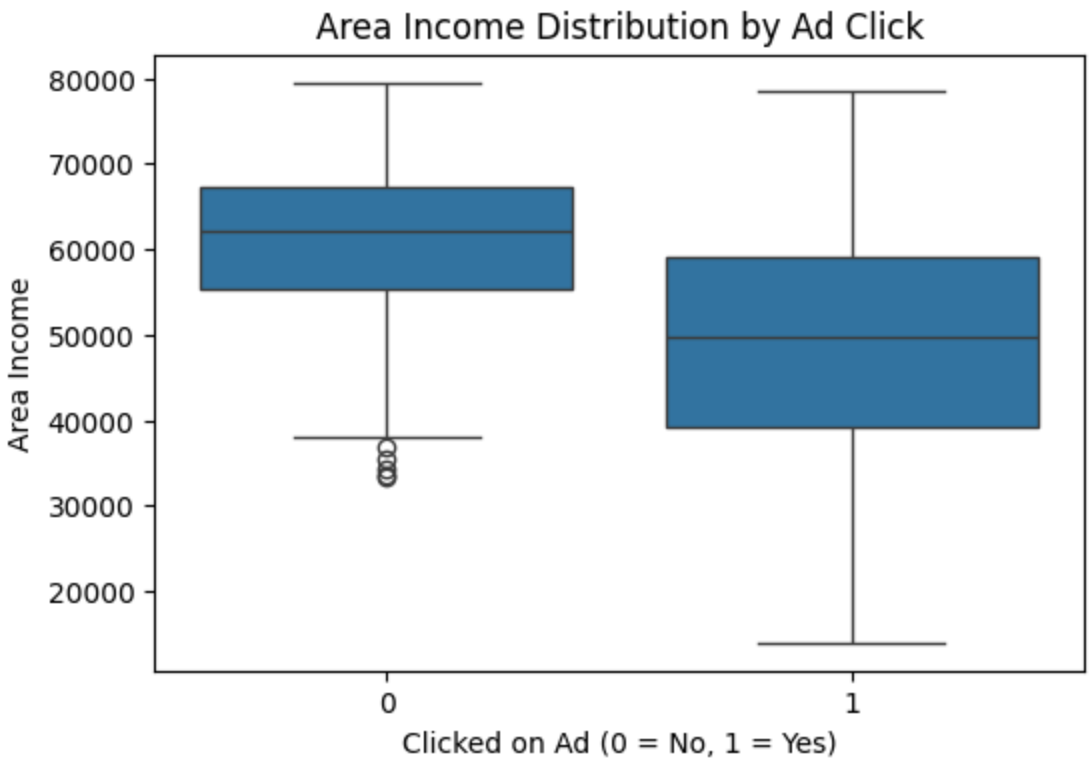
In [23]:
```
# Correlation Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```



Daily Internet Usage vs Clicked on Ad shows a negative correlation of -0.79, which is the strongest relationship in the dataset. Users who spend more time on the internet daily are less likely to click ads because they understand the ad tricks, whiles Users with lower internet usage tend to click ads more.

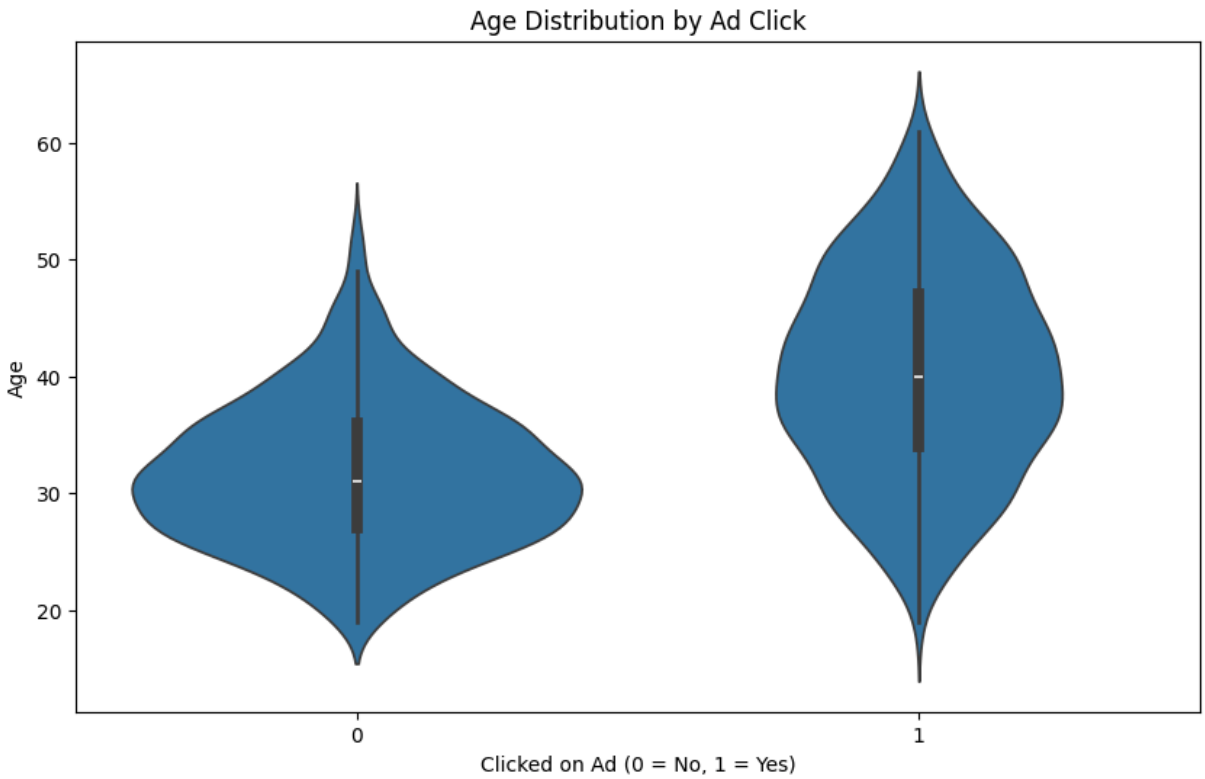
More so, Daily Time Spent on Site vs Clicked on Ad is -0.74 also a strong negative correlation. People who spend more time browsing the site tend to click ads less, when people who spend less time on the site click ads more often. So, heavy users may ignore ads.

In [24]:
```
# Area Income vs Ad Click
# To check people who click ads tend to have income level
plt.figure(figsize=(6,4))
sns.boxplot(x='Clicked on Ad', y='Area Income', data=df)
plt.title('Area Income Distribution by Ad Click')
plt.xlabel('Clicked on Ad (0 = No, 1 = Yes)')
plt.ylabel('Area Income')
plt.show()
```



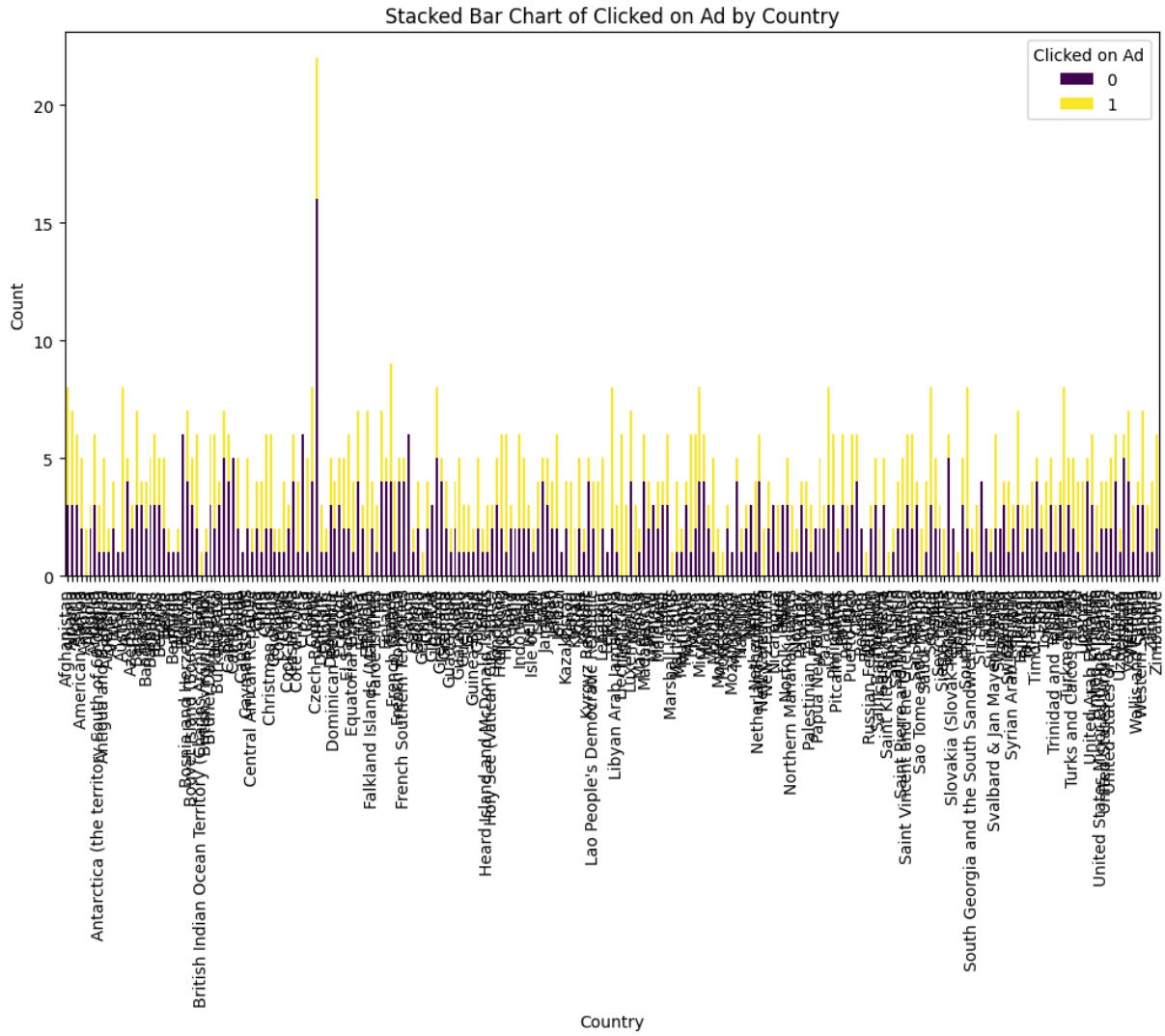
The older ushers who tend to click the ad come from lower-income groups compared to those who did not click the ad, suggesting that age and area income may play a critical role in influencing ad-click behavior.

In [25]:
```
# Violin Plot for Age and Clicked on Ad
plt.figure(figsize=(10,6))
sns.violinplot(x='Clicked on Ad', y='Age', data=df)
plt.title('Age Distribution by Ad Click')
plt.xlabel('Clicked on Ad (0 = No, 1 = Yes)')
plt.ylabel('Age')
plt.show()
```

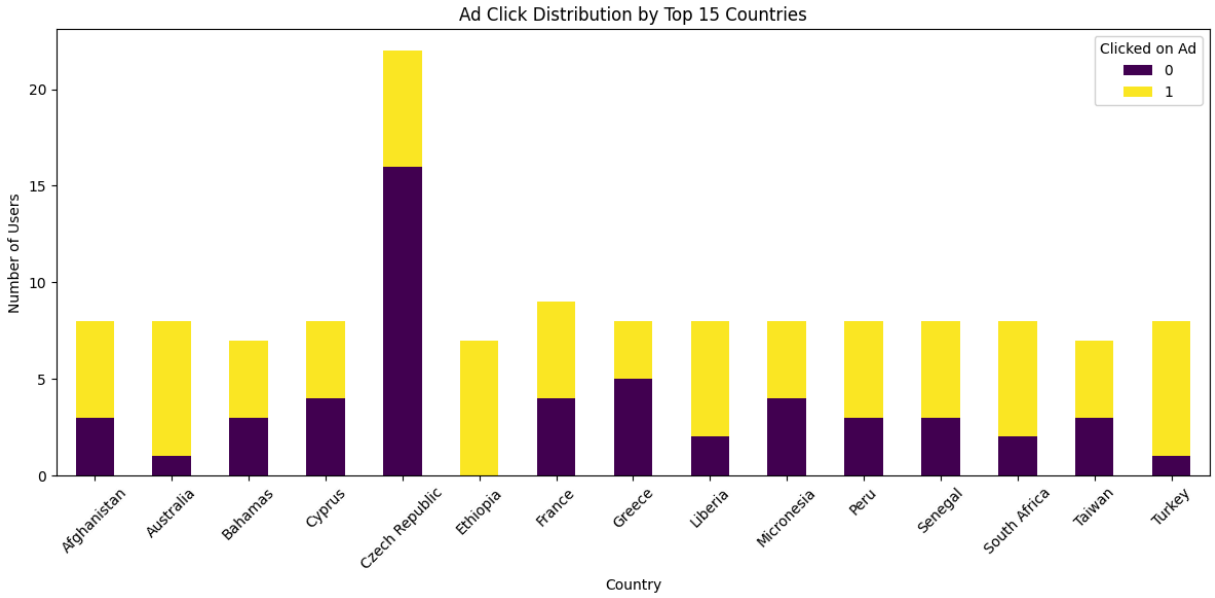


Clicking of Ad tend to be age factor here, the older people from the age group in their late 35 to early 45 years clicked on ads tend to be older, while people who did not click on ads are generally younger, with most ages concentrated around the late 25 to early 35.

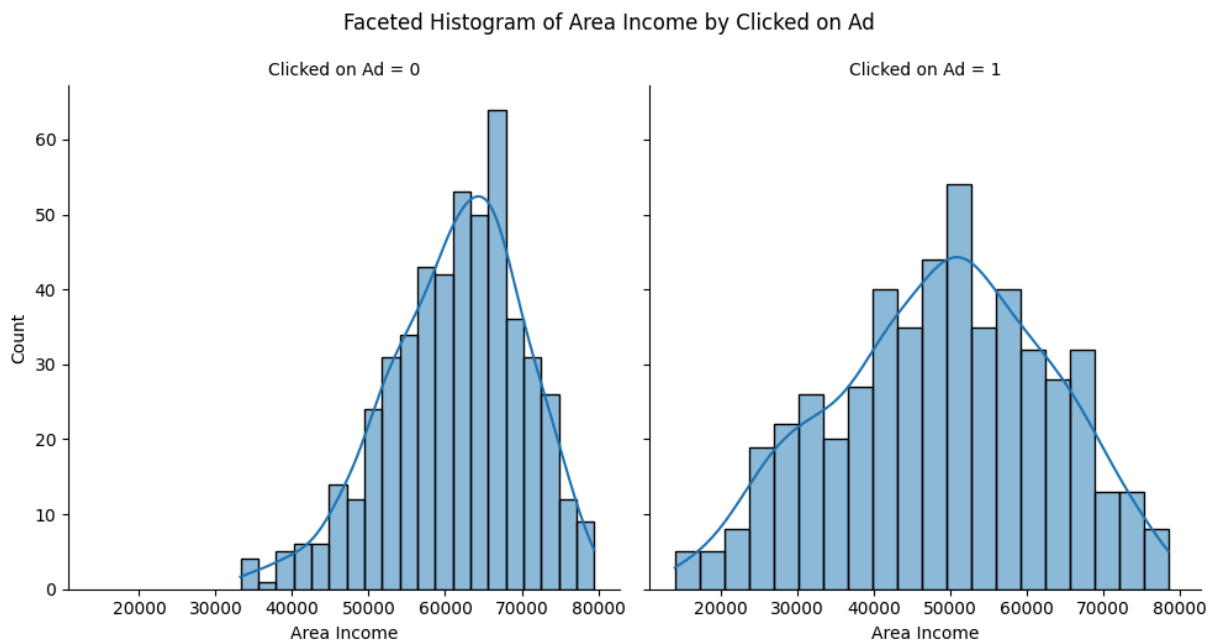
In [26]:
```
# Stacked Bar Chart of Clicked on Ad by County
# This chart shows the count of ad clicks by city.

city_counts = df.groupby(['Country', 'Clicked on Ad']).size().unstack(fill_value=0)
city_counts.plot(kind='bar', stacked=True, figsize=(12, 6), colormap='viridis')
plt.title('Stacked Bar Chart of Clicked on Ad by Country')
plt.xlabel('Country')
plt.ylabel('Count')
plt.show()
```



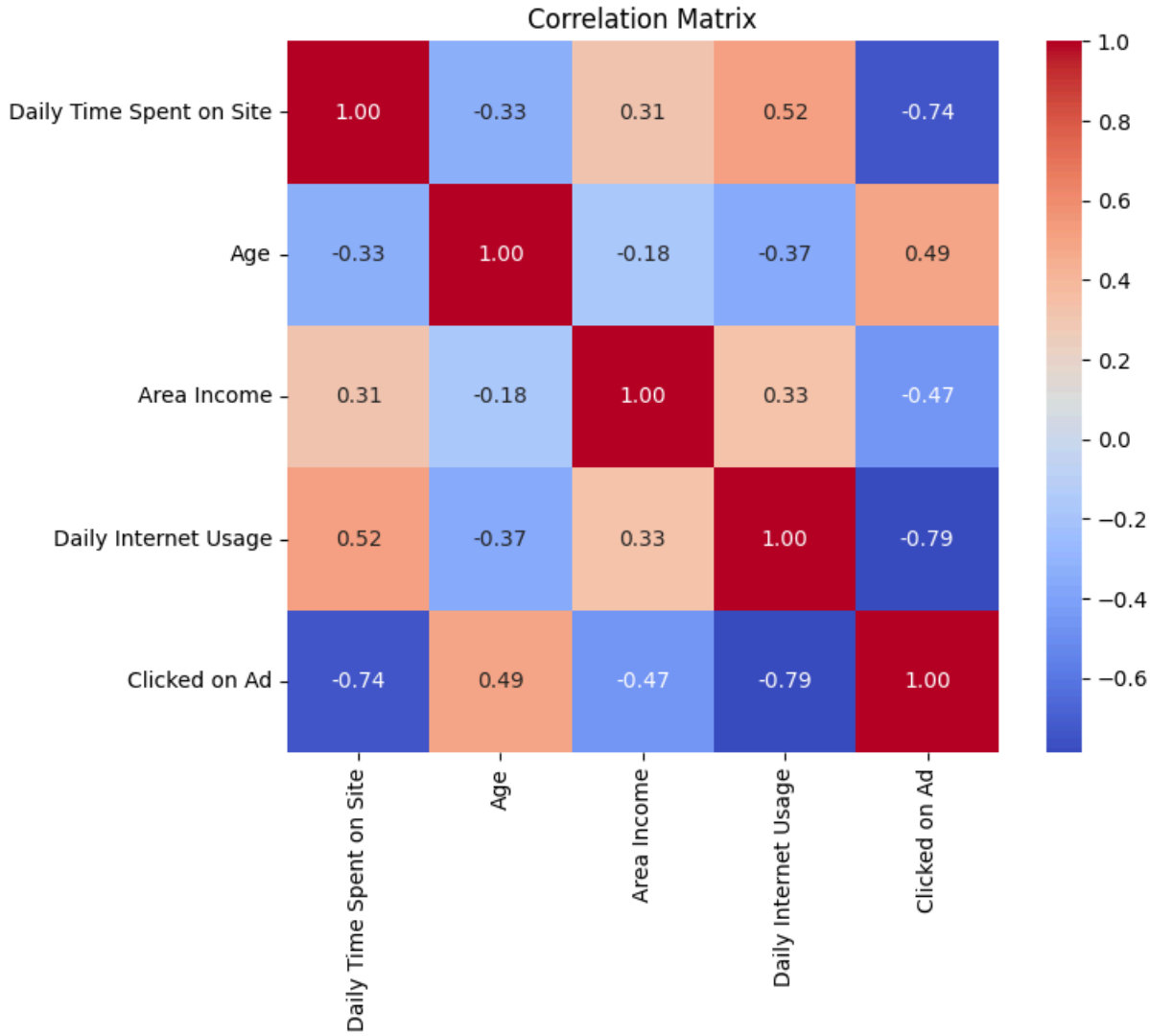
In [27]:
```
# Select top 15 countries with the most observations
top_countries = df['Country'].value_counts().nlargest(15).index
df_top = df[df['Country'].isin(top_countries)]
country_counts = df_top.groupby(['Country', 'Clicked on Ad']).size().unstack(fill_v
country_counts.plot(kind='bar', stacked=True, figsize=(12,6), colormap='viridis')
plt.title('Ad Click Distribution by Top 15 Countries')
plt.xlabel('Country')
plt.ylabel('Number of Users')
plt.xticks(rotation=45)
plt.legend(title='Clicked on Ad')
plt.tight_layout()
plt.show()
```



In [28]:
```
# To identify potential bias in the 'Area Income' column of your dataset:
g = sns.FacetGrid(df, col="Clicked on Ad", height=5, aspect=1)
g.map(sns.histplot, "Area Income", bins=20, kde=True)
g.set_axis_labels("Area Income", "Count")
g.set_titles("Clicked on Ad = {col_name}")
g.fig.suptitle('Faceted Histogram of Area Income by Clicked on Ad', y=1.05)
plt.show()
```

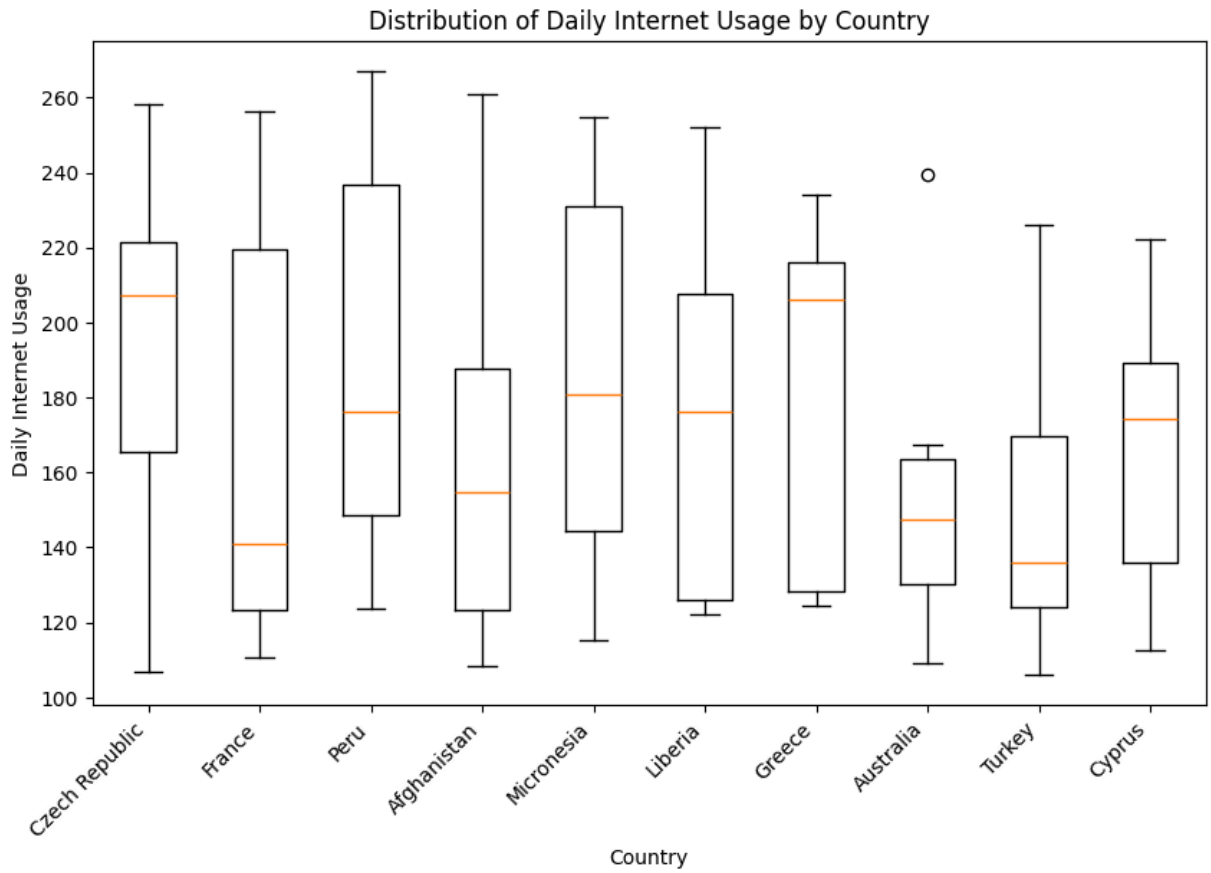


In [29]:
```
# Correlation Matrix
# get the numerical columns from the dataset
num_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()
plt.figure(figsize=(8, 6))
correlation_matrix = df[num_cols].corr()
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', square=True
plt.title('Correlation Matrix')
plt.show()
```



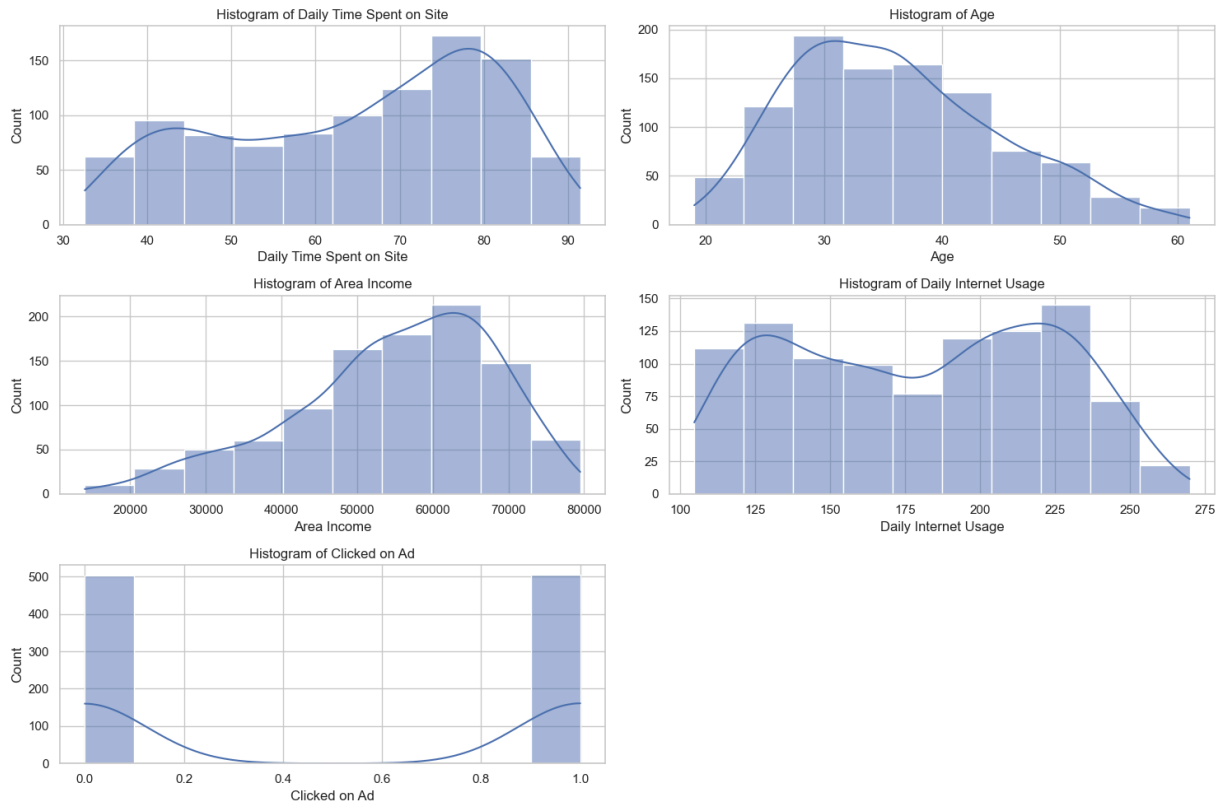
This shows the relationships between key variables. The strongest correlations amongst the variables with ad clicking behavior are Daily Internet Usage -0.79,it demonstrates that heavy internet users are often more aware of advertising tactics and tend to ignore ads and Daily Time Spent on Site -0.74 negatibe correlation indicating that users who spend more time online are less likely to click advertisements. However, Age shows a moderate positive correlation 0.49, suggesting that older users tend to click ads more frequently whiles the younger users are more skeptical on ads. Area Income has a moderate negative correlation -0.47, this implies that users from higher-income areas are less likely to engage with ads.

In [31]:
```
# Distribution of Daily Internet Usage by Country
# get the top 10 countries by occurances
top_countries = df['Country'].value_counts().nlargest(10).index
plt.figure(figsize=(10, 6))
plt.boxplot([df[df['Country'] == country]['Daily Internet Usage'] for country in to
plt.xlabel('Country')
plt.ylabel('Daily Internet Usage')
plt.title('Distribution of Daily Internet Usage by Country')
____ = plt.xticks(rotation=45, ha='right')
```



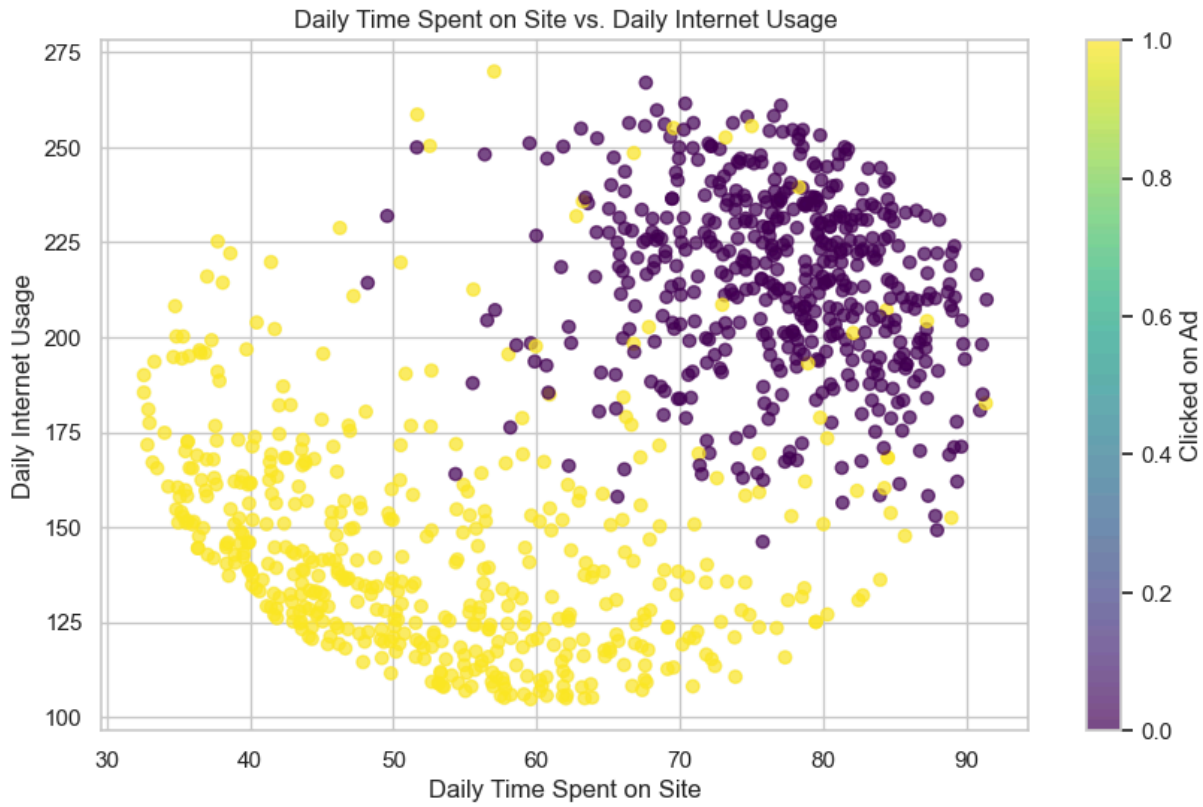
This illustrates the distribution of daily internet usage across the ten most represented countries in the dataset. The Czech Republic and Greece exhibit higher median internet usage, while Turkey and Afghanistan show lower median values. The variability of internet usage differs across countries, with some displaying wider distributions, indicating diverse user behavior. Additionally, a few outliers are observed, suggesting unusually high internet usage among certain individuals.

In [32]:
```
# Historgram for numerical features
sns.set(style="whitegrid")
num_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()
plt.figure(figsize=(15, 10))
for i, col in enumerate(num_cols):
    plt.subplot(3, 2, i + 1)
    sns.histplot(df[col].dropna(), bins=10, kde=True)
    plt.title(f'Histogram of {col}')
plt.tight_layout()
plt.show()
```



In [33]:

```
# Daily Time Spent on Site vs. Daily Internet Usage

plt.figure(figsize=(10, 6))

plt.scatter(
    x = df['Daily Time Spent on Site'],
    y = df['Daily Internet Usage'],
    c = df['Clicked on Ad'],
    cmap='viridis',
    alpha=0.7)
plt.xlabel('Daily Time Spent on Site')
plt.ylabel('Daily Internet Usage')
plt.title('Daily Time Spent on Site vs. Daily Internet Usage')

_ = plt.colorbar(label='Clicked on Ad')

plt.show()
```



Less Internet usage and more likelihood of clicking the add.

This pattern suggests that internet usage behavior strongly influences ad engagement. The yellow color represents the user clicked on an advertisement.Users with higher internet usage and more time spent on the website are less likely to click advertisements, while users with lower internet usage and less time on the website are more likely to click ads

Milestone 3: Predictive Modeling

In this step, we will

- encode categorical variables
- select relevant features
- split the data into train/test
- scale the data
- train and test the model
- evaluate the model performance.

In [34]:
```
# creating a copy of the the original dataset
df2 = df.copy()
```

In [35]:
```
#Import the necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Categorical variables such as 'City' and 'Country' need to be converted into numerical representations for machine learning models. We use one-hot encoding to achieve this, which creates binary columns for each category. Learn more about One-Hot Encoding here.

This process converts each unique value in a categorical column into a new binary column. For example, if there are three cities (New York, Los Angeles, Chicago), three new columns will be created (City_New York, City_Los Angeles, City_Chicago), with 1 indicating the presence of that city and 0 otherwise.

drop_first=True: This argument prevents multicollinearity by dropping one category from each set of dummy variables.

In [37]:
```
# Encode categorical variables
df2 = pd.get_dummies(df2, columns=['City', 'Country'], drop_first=True)
df2
```

Out[37]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	Gender	Timestamp	Clicked on Ad	Hour
0	68.95	35.0	61833.90	256.09	Cloned 5thgeneration orchestration	Female	2016-03-27 00:53:00	0	0
1	NaN	31.0	68441.85	193.77	Monitored national standardization	Male	2016-04-04 01:39:00	0	1
2	69.47	26.0	59785.94	236.50	Organic bottom-line service-desk	Female	2016-03-13 20:35:00	0	20
3	74.15	29.0	54806.18	245.89	Triple-buffered reciprocal time-frame	Male	2016-10-01 02:31:00	0	2
4	68.37	35.0	73889.99	225.58	Robust logistical utilization	Female	2016-03-06 03:36:00	0	3
...	...	...	...	...	...	...	...	...	...
1004	72.97	30.0	71384.57	208.58	Fundamental modular algorithm	Male	2016-11-02 21:49:00	1	21
1005	51.30	45.0	67782.17	134.42	Grass-roots cohesive monitoring	Male	2016-04-22 02:07:00	1	2
1006	51.63	51.0	42415.72	120.37	Expanded intangible solution	Male	2016-01-02 17:24:00	1	17
1007	55.55	19.0	41920.79	187.95	Proactive bandwidth-monitored policy	Female	2016-03-24 02:35:00	0	2
1008	45.01	26.0	29875.80	178.35	Virtual 5thgeneration emulation	Female	2016-03-06 21:43:00	1	21

1009 rows × 1207 columns

Next, we **select the features** that will be used in our predictive model.

And finally, we prepare the feature matrix `X` and target vector `y`.

In [38]:
```
# Select features for modeling
base_features = ['Daily Time Spent on Site', 'Age', 'Area Income', 'Daily Internet
encoded_cols = [col for col in df2.columns if col.startswith(('City_', 'Country_'))
features = base_features + encoded_cols
X = df2[features]
y = df2['Clicked on Ad']
```

In [39]:
```
#Checking missing values if there is any
X.isna().sum()
```

Out[39]:
```
Daily Time Spent on Site     4
Age                          0
Area Income                  0
Daily Internet Usage         4
City_Adamsbury               0
                            ..
Country_Wallis and Futuna    0
Country_Western Sahara       0
Country_Yemen                0
Country_Zambia               0
Country_Zimbabwe             0
Length: 1200, dtype: int64
```

It appears that Daily Time Spent on Site has 4 missing values and Daily Internet Usage also have 4 missing values. In this case, we will check the percentage of the missing values in the data before fixing them

In [40]:
```
#Checking the percentage of missing values
missing_percentage = (df.isna().sum() / len(df)) * 100
print(missing_percentage)
```

```
Daily Time Spent on Site    0.396432
Age                         0.000000
Area Income                 0.000000
Daily Internet Usage        0.396432
Ad Topic Line               0.000000
City                        0.000000
Gender                      0.000000
Country                     0.000000
Timestamp                   0.000000
Clicked on Ad               0.000000
Hour                        0.000000
Day                         0.000000
Month                       0.000000
dtype: float64
```

The percentage of missing values in both Daily Time Spent on Site and Daily Internet Usage together is less than 1 percent, and very small. This looks promising for fixing because the percentage is extremely low.

In [41]:
```
#Fixing the missing values with Mean imputation
X['Daily Time Spent on Site'].fillna(X['Daily Time Spent on Site'].mean(), inplace=
X['Daily Internet Usage'].fillna(X['Daily Internet Usage'].mean(), inplace=True)
```

In [42]:
```
#Checking the missing values again
X.isnull().sum()
```

Out[42]:
```
Daily Time Spent on Site     0
Age                          0
Area Income                  0
Daily Internet Usage         0
City_Adamsbury               0
                            ..
Country_Wallis and Futuna    0
Country_Western Sahara       0
Country_Yemen                0
Country_Zambia               0
Country_Zimbabwe             0
Length: 1200, dtype: int64
```

This looks good and ready for modeling

Splitting the data:
We use `train_test_split()` from the `sklearn.model_selection` module to split the data into training and testing sets. The `test_size` parameter should be set to `0.2`, meaning 20% of the data will be used for testing, while 80% will be used for training. A `random_state` is set to ensure reproducibility.

In [43]:
```
# Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

Scaling the data:
Next, we scale the features using `StandardScaler` from `sklearn.preprocessing`. Standardization rescales the data so that each feature has a mean of `0` and a standard deviation of `1`. This is particularly important when using models that rely on distance metrics (e.g., logistic regression, k-nearest neighbors).

Learn more about standardization here.

In [45]:
```
# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [46]:
```
# Checking data after split
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (807, 1200)
X_test shape: (202, 1200)
y_train shape: (807,)
y_test shape: (202,)
```

Train and Test

Now, we use the `LogisticRegression` model to fit on our train data, and then make predictions on the test data using `model.predict`.

In [47]:
```
from sklearn.linear_model import LogisticRegression

# Logistic Regression Model

# Train logistic regression model
lr_model = LogisticRegression(random_state=42)
lr_model.fit(X_train_scaled, y_train)

y_pred = lr_model.predict(X_test_scaled)
```

In [49]:
```
# Evaluate the model
# evaluate accuracy score
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

print("Accuracy:")
accuracy_score(y_test, y_pred)
```

```
Accuracy:
```

Out[49]:
```
0.9504950495049505
```

Out of 202 total predictions of the test dataset, about 192 were correct with 95% accuracy

In [50]:
```
#Get confusion matrix
print("Confusion Matrix:")
confusion_matrix(y_test, y_pred)
```

```
Confusion Matrix:
```

Out[50]:
```
array([[93,  3],
       [ 7, 99]], dtype=int64)
```

93 users who did not click the advertisement and 99 users who clicked it. Only 3 false positives and 7 false negatives were recorded, meaning the model made very few incorrect predictions. Overall, this suggests that the model is highly accurate and effective at identifying users who are likely to engage with advertisements.

In [51]:
```
# Get classification report
print("Classification Report:")
classification_report(y_test, y_pred)
```

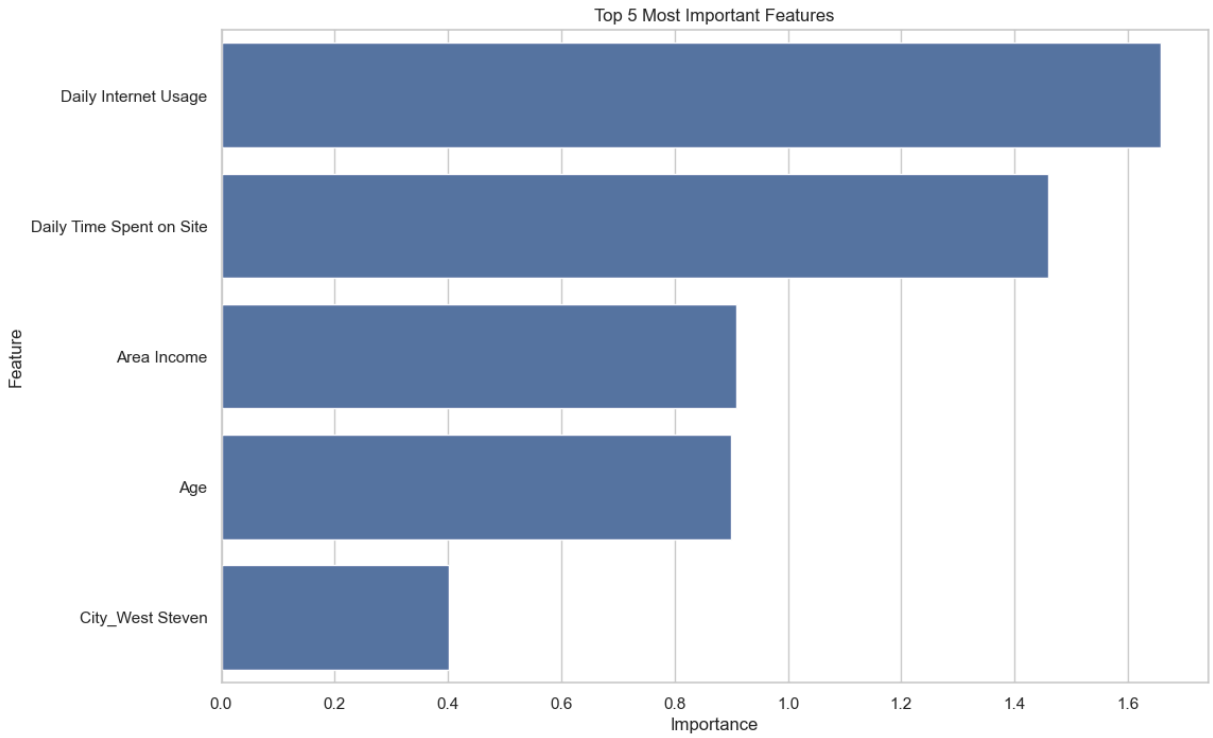
```
Classification Report:
```

Out[51]:
```
'              precision    recall  f1-score   support\n\n           0       0.93
0.97      0.95        96\n           1       0.97      0.93      0.95       106\n
\n    accuracy                           0.95       202\n   macro avg       0.95
0.95      0.95       202\nweighted avg       0.95      0.95      0.95       202\n'
```

The model performed very well, achieving an overall accuracy of 95%. The precision and recall values for both classes are above 0.93, which suggests that the model is effective at

identifying users who click on advertisements as well as those who do not. In addition, the F1-score of 0.95 for both groups shows that the model maintains a good balance between precision and recall. Overall, these results indicate that the model is reliable and performs consistently when predicting whether a user will click on an advertisement.

In [52]:
```
# Feature importance
feature_importance = pd.DataFrame({'Feature': features, 'Importance': abs(lr_model.
feature_importance = feature_importance.sort_values('Importance', ascending=False)

plt.figure(figsize=(12, 8))
sns.barplot(x='Importance', y='Feature', data=feature_importance.head(5))
plt.title('Top 5 Most Important Features')
plt.show()
```



Summary:

The logistic regression model achieved 95% accuracy, indicating strong predictive performance in identifying whether users click on advertisements. The confusion matrix shows a high number of correct predictions with very few misclassifications. Precision, recall, and F1-scores for both classes are around 0.95, demonstrating that the model is both accurate and well-balanced in predicting ad click behavior